ETP Xavier	MÒDUL 0613 – Desenvolupament web en entorn servidor			
	Tema 5 – Activitat 1: Consum i integració d'una API externa en una aplicació web			
	DATA entrega	26/04/2026 23:55	GRUP	2n
	Nom professor	Carlos Vacas		

OBJECTIU GENERAL

Desenvolupar una aplicació web capaç de:

- consumir una API externa
- utilitzar diferents mètodes HTTP
- processar dades JSON
- integrar dades externes amb una aplicació pròpia

RESULTATS D'APRENENTATGE QUE COBREIX

RA7 (Serveis web)

- consumir API (GET, POST, PUT, DELETE)
- entendre HTTP i JSON
- verificar funcionament amb Postman
- gestionar respostes

RA9 (Aplicacions híbrides)

- reutilitzar informació externa
- integrar dades en una app
- combinar dades (externes + pròpies)
- crear funcionalitat útil

ESTRUCTURA DE L'ACTIVITAT

PART 1 — CONSUM API AMB PHP (cURL)

OBJECTIU

Construir un client PHP que interactuï amb:
https://jsonplaceholder.typicode.com/posts

REQUISITS
Crear un projecte amb:

```
/api-client/
├── config.php
├── curl_client.php
├── index.php
```

FUNCIONALITATS OBLIGATÒRIES
S'ha d'implementar:

1. GET (llistar posts)

- mostrar mínim 5 resultats

ssabir94's Workspace | Search | Invite | Upgrade

GET Get data | GET Get data | No environment

My Collection > Get data | Save | Share

GET https://jsonplaceholder.typicode.com/posts Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK | 168 ms | 7.99 KB | Save Response

{} JSON | Preview | Visualize


```
1  [
2      {
3          "userId": 1,
4          "id": 1,
5          "title": "sunt aut facere repellat provident occaecati excepturi optio
               reprehenderit",
6          "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et
               cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem
               sunt rem eveniet architecto"
7      },
8      {
9          "userId": 1,
10         "id": 2,
11         "title": "qui est esse",
12         "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor
               beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil
               molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi
               nulla"
```


2. GET per ID

- obtenir un post concret

GET https://jsonplaceholder.typicode.com/posts/1 Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

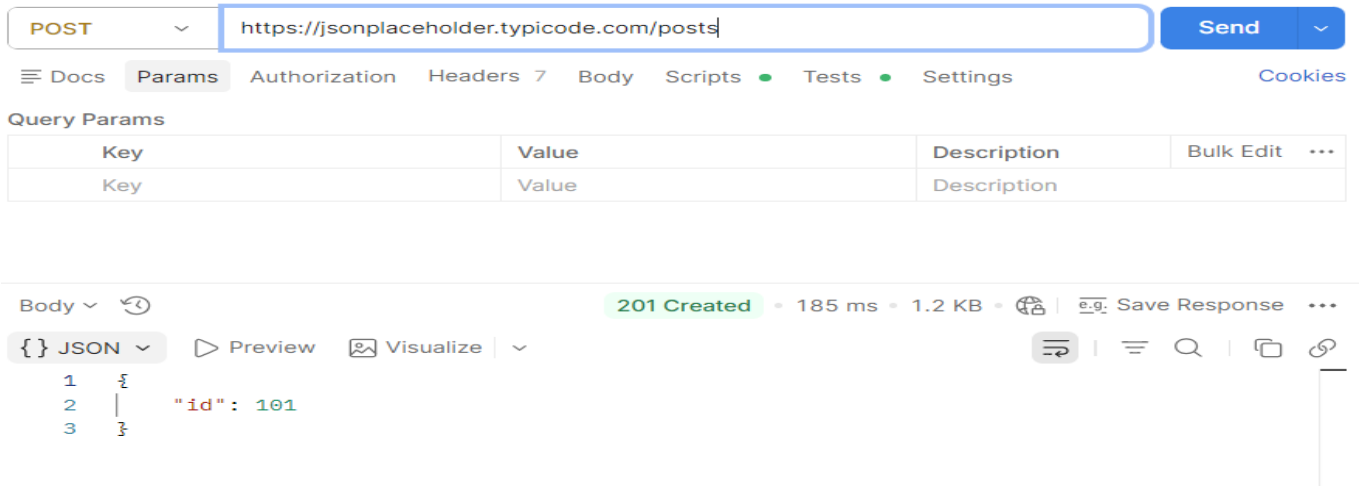
Body | 200 OK | 63 ms | 1.27 KB | Save Response

{} JSON | Preview | Visualize

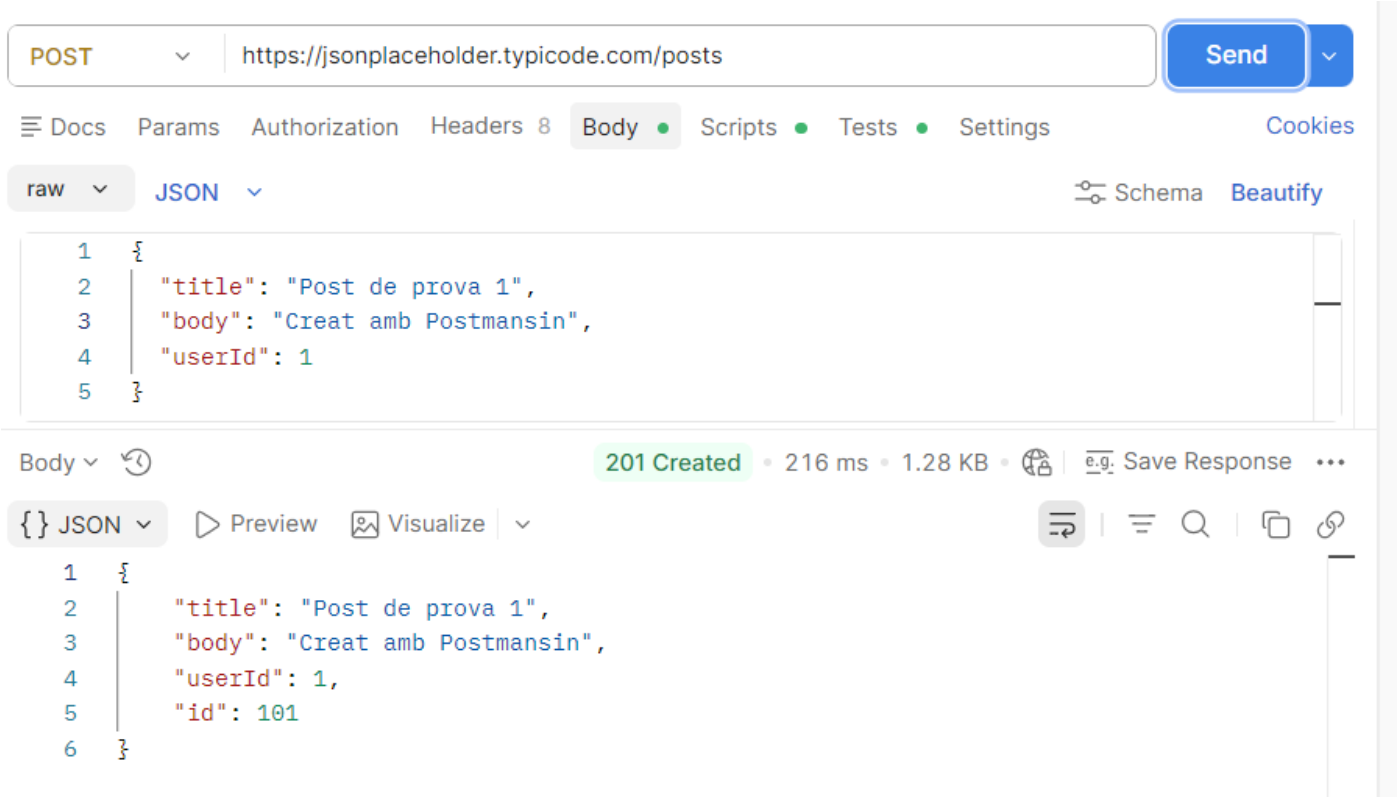

```
1  {
2      "userId": 1,
3      "id": 1,
4      "title": "sunt aut facere repellat provident occaecati excepturi optio
           reprehenderit",
5      "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et
           cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt
           rem eveniet architecto"
6  }
```

3. POST

- enviar JSON
- mostrar resposta



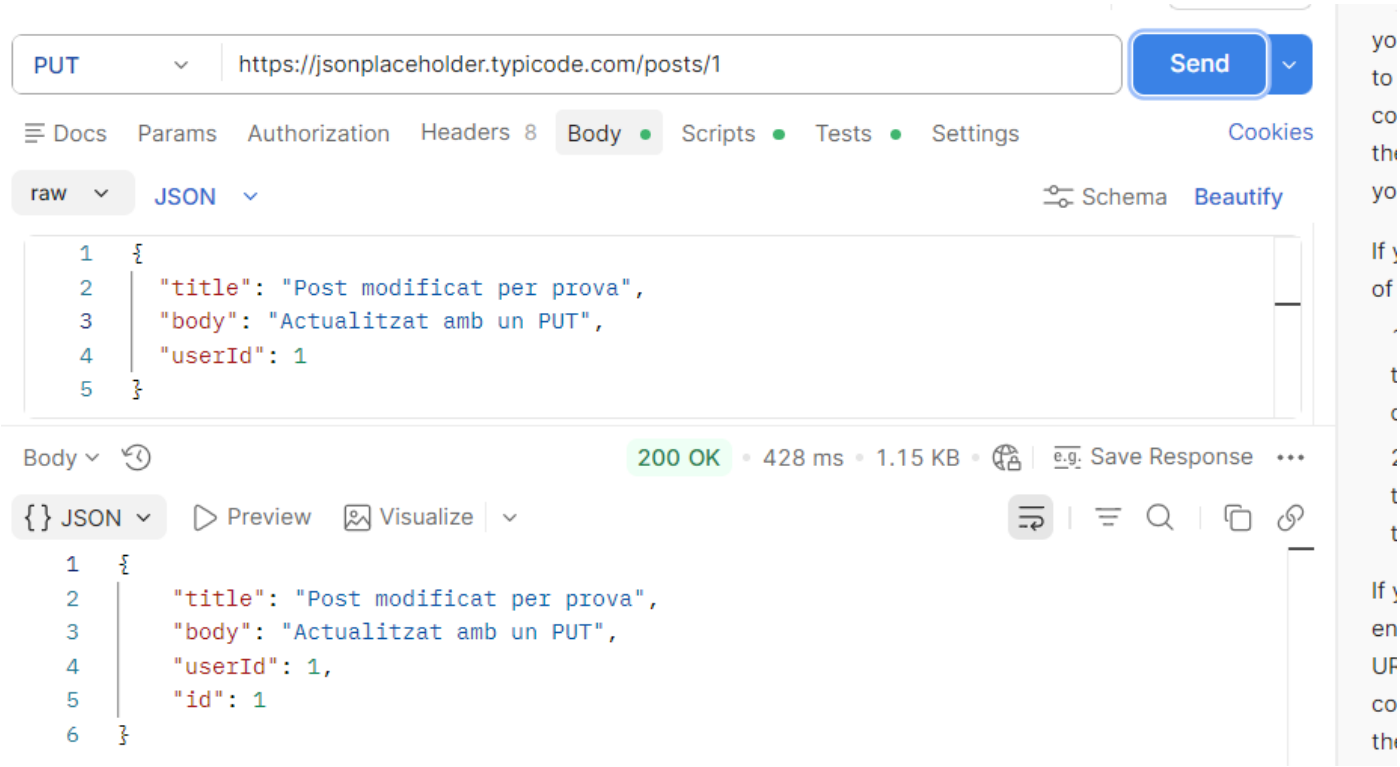
Vaig a Body, i dintre de JSON poso el codi de prova:



Ara si! :)

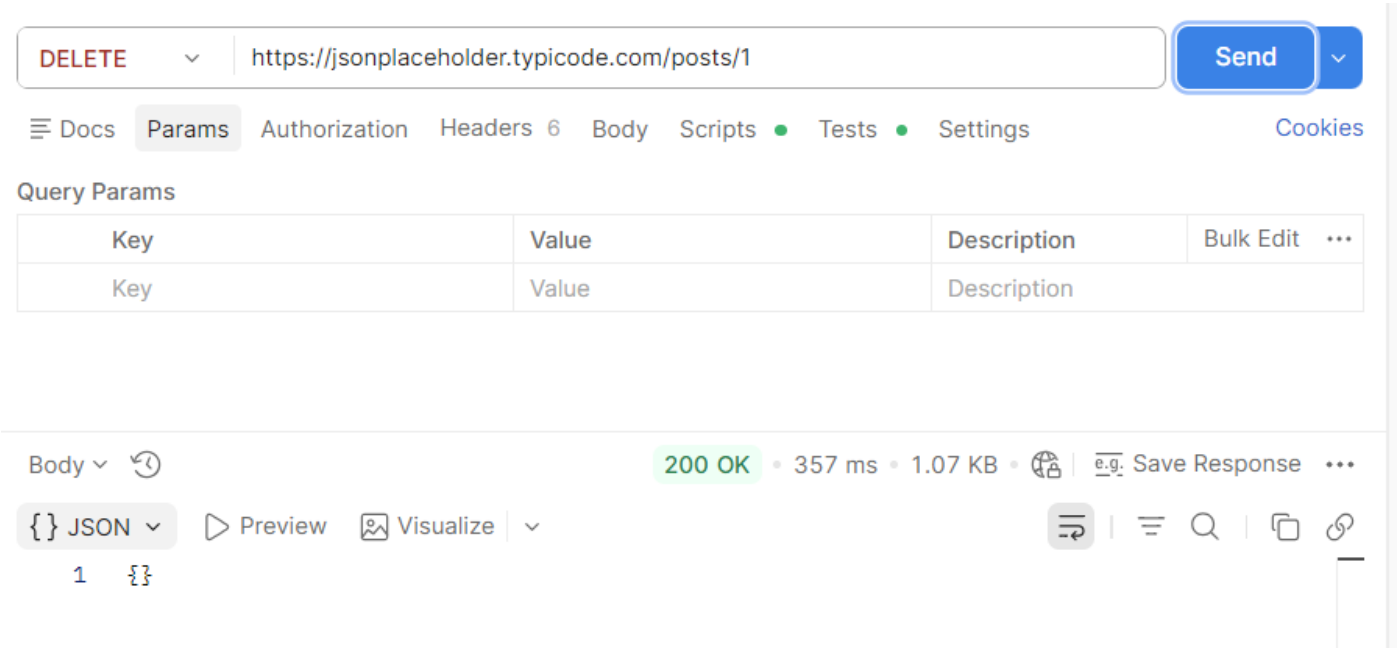
4. PUT

- modificar un registre



5. DELETE

- eliminar un registre

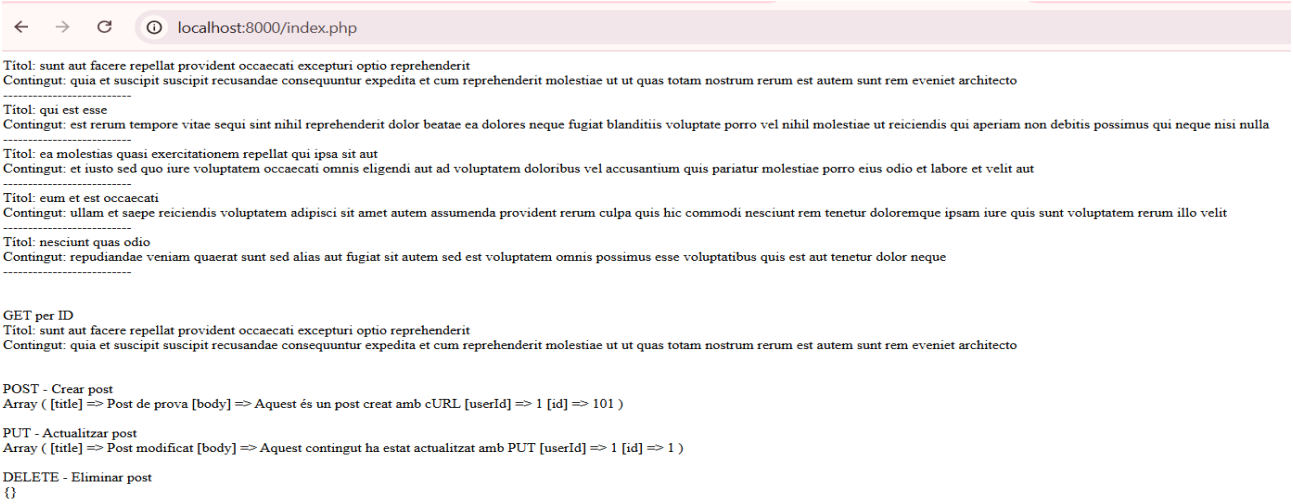


CONDICIONS

- NO es permeten formularis HTML
- totes les proves amb **Postman**
- codi modular (funcions separades)

EVIDÈNCIES

- codi funcional



- captures Postman

✔ (Captures anteriors) :D

- explicació de cada mètode

  - GET (llistar dades)

El mètode GET s'utilitza per obtenir informació d'una API. En aquest cas, s'ha fet servir per recuperar una llista de posts del servidor. La resposta es rep en format JSON i es converteix a un array de PHP per poder-la mostrar.

  - GET per ID

El mètode GET també permet obtenir un recurs concret indicant el seu identificador a la URL. En aquest cas, s'ha utilitzat per recuperar un únic post a partir del seu ID.

  - POST (crear dades)

El mètode POST s'utilitza per enviar dades a una API amb l'objectiu de crear un nou recurs. En aquest cas, s'ha enviat informació en format JSON per crear un nou post. La resposta retorna les dades enviades juntament amb un identificador nou.

  - PUT (actualitzar dades)

El mètode PUT s'utilitza per modificar un recurs existent. En aquest cas, s'ha utilitzat per actualitzar un post indicant el seu ID i enviant les noves dades en format JSON.

  - DELETE (eliminar dades)

El mètode DELETE s'utilitza per eliminar un recurs del servidor. En aquest cas, s'ha fet servir per eliminar un post a partir del seu ID. La resposta acostuma a ser buida o molt simple.

PART 2 — CONSUM AMB LARAVEL

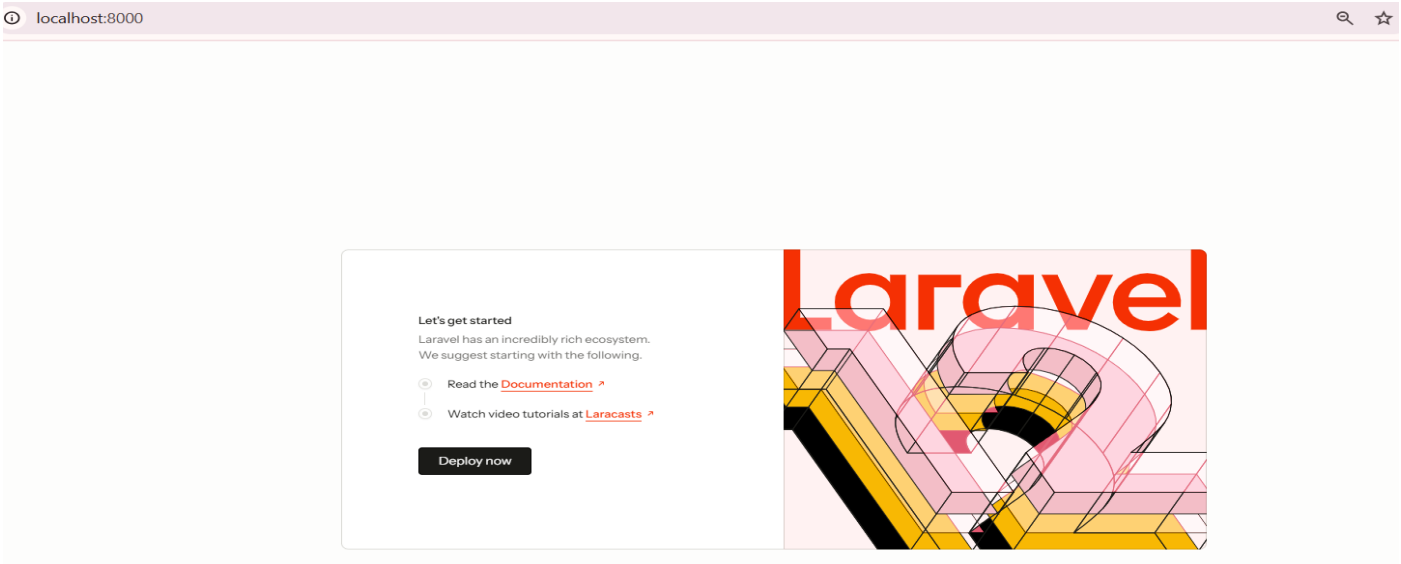
OBJECTIU
Repetir el consum anterior utilitzant Laravel

```
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1>composer create-project laravel/laravel consum-api-lara
vel
Creating a "laravel/laravel" project at "./consum-api-laravel"
Cannot use laravel/laravel's latest version v13.3.0 as it requires php ^8.3 which is not satisfied by your platform.
Installing laravel/laravel (v12.12.2)
  - Installing laravel/laravel (v12.12.2): Extracting archive
Created project in C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
```

Provem-ho!

```
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>php artisan serve

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

Tot ok! :




```
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>php artisan make:controller ApiContr
oller

  INFO  Controller [C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel\app\Http\Contro
llers\ApiController.php] created successfully.
```

REQUISITS

- projecte Laravel funcional
- controlador API

FUNCIONALITATS

- GET (vista Blade)

ApiController.php

U05EX1 > consum-api-laravel > app > Http > Controllers > ApiController.php > PHP > ApiController > getPosts()

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Support\Facades\Http;

0 references | 0 implementations
class ApiController extends Controller
{
    // Aquesta funció obté els posts de l'API
    0 references | 0 overrides
    public function getPosts()
    {
        // Faig la petició GET a l'API externa
        $response = Http::get('https://jsonplaceholder.typicode.com/posts');

        // Converteixo la resposta JSON a array
        $posts = $response->json();

        // Retorno les dades
        return $posts;
    }
}
```

Ara modifiquem *web.php* :

U05EX1 > consum-api-laravel > routes > web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ApiController;

Route::get('/', function () {
    return view('welcome');
});


Route::get('/posts', [ApiController::class, 'getPosts']);
```

El resultat:

localhost:8000/posts

Aplica estils de format


```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse"
```

Ara anem amb el BLADE. Creo "*posts.blade.php*":

```
U05EX1 > consum-api-laravel > resources > views > posts.blade.php
<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="UTF-8">
    <title>Posts API</title>
</head>
<body>

    <h1>Llistat de posts</h1>

    @foreach($posts as $post)
        <h2>{{ $post['title'] }}</h2>
        <p>{{ $post['body'] }}</p>
        <hr>
    @endforeach

</body>
</html>
```

Un cop creat, modifico el retorn de dades d' *ApiController*:

```
    // Retorno les dades
    return view('posts', compact('posts'));
}
```

Per últim, fem comprovació de que tot ha sortit bé:

localhost:8000/posts

Llistat de posts

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

---

qui est esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla

---

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut

---

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit

---

nesciunt quas odio

repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

---

Ara ja podem veure la llista 😊

- POST

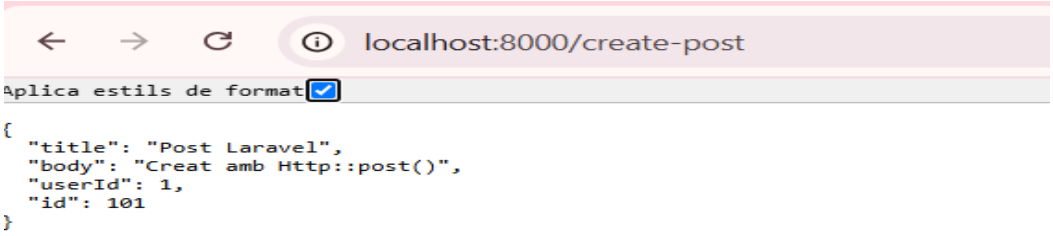
Ara toca modificar de nou *ApiController*. Afegeixo nova funcionalitat:

```
// Aquesta funció crea un nou post amb POST
0 references | 0 overrides
public function createPost()
{
    // Envio dades a l'API
    $response = Http::post('https://jsonplaceholder.typicode.com/posts', [
        'title' => 'Post Laravel',
        'body' => 'Creat amb Http::post()',
        'userId' => 1
    ]);

    // Retorno la resposta en JSON
    return $response->json();
}
```

Afegeixo això a *web.php*:

```
Route::get('/create-post', [ApiController::class, 'createPost']);
```

Aviam que surt...



Genial, això vol dir que funciona ^^.

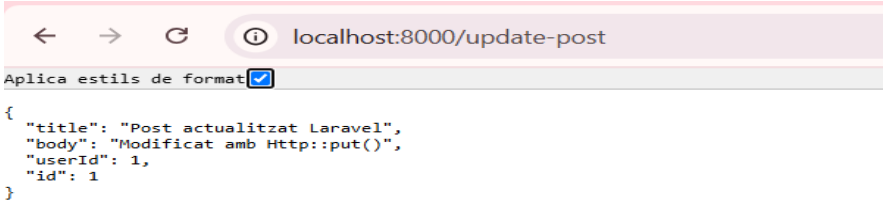
- PUT

Tornem a editar *ApiController*:

```
// Aquesta funció actualitza un post amb PUT
0 references | 0 overrides
public function updatePost()
{
    // Envio dades per modificar el post amb ID = 1
    $response = Http::put('https://jsonplaceholder.typicode.com/posts/1', [
        'title' => 'Post actualitzat Laravel',
        'body' => 'Modificat amb Http::put()',
        'userId' => 1
    ]);

    // Retorno la resposta en JSON
    return $response->json();
}
```

```
Route::get('/update-post', [ApiController::class, 'updatePost']);
```

Ara el resultat:



- DELETE

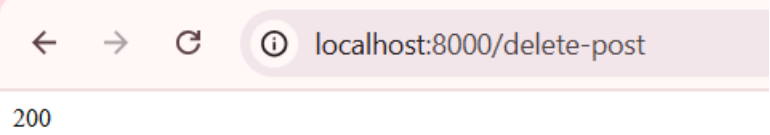
Mateixes passes que al anteriors:

```
// Aquesta funció elimina un post amb DELETE
0 references | 0 overrides
public function deletePost()
{
    // Faig la petició DELETE al post amb ID = 1
    $response = Http::delete('https://jsonplaceholder.typicode.com/posts/1');

    // Retorno el codi d'estat HTTP
    return $response->status();
}
```

De nou, web.php:

```
Route::get('/delete-post', [ApiController::class, 'deletePost']);
```

Última comprovació:



Ok! :D

IMPORTANT

- utilitzar Http::

- mostrar resultats

PART 3 — INTEGRACIÓ (APLICACIÓ HÍBRIDA) (RA9)

OBJECTIU

Construir una aplicació que integri API + base de dades pròpia

REQUISITS

Aplicació Laravel amb:

- consum API posts
- sistema de favorits (BD)

```
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>php artisan make:model Favorit -m

  INFO  Model [C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel\app\Models\Favorit.p
hp] created successfully.

  INFO  Migration [C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel\database\migrati
ons\2026_04_29_211307_create_favorits_table.php] created successfully.


C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>
```

Fem migració, i creem taula de Favorits:

```
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>php artisan migrate

  INFO  Running migrations.

  2026_04_29_211307_create_favorits_table ..................................................................... 268.10ms DONE

C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>
```

Crear Controlador:

```
C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>php artisan make:controller PostCont
roller

  INFO  Controller [C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel\app\Http\Contro
llers\PostController.php] created successfully.


C:\Users\HP\Documents\DAW 2N\DES. WEB ENT. SERVIDOR\3a EV\U05EX1\consum-api-laravel>
```

FUNCIONALITATS

1. Mostrar posts (API)

```php
0 references | 0 implementations
class PostController extends Controller
{
    0 references | 0 overrides
    public function index()
    {
        // Obtenir posts de l'API externa
        $response = Http::get('https://jsonplaceholder.typicode.com/posts');
        $posts = $response->json();

        // Obtenir els IDs dels favorits guardats a la BD
        $favorits = Favorit::pluck('post_id')->toArray();

        // Enviar les dades a la vista
        return view('posts', compact('posts', 'favorits'));
    }
}
```

A *web.php* afegim nova ruta:

```php
Route::get('/posts', [PostController::class, 'index']);
```

Fem php artisan serve i provem:

localhost:8000/posts

Llistat de posts

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

---

qui est esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla

---

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut

---

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit

---

nesciunt quas odio

repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

---

Pel que sembla, funciona ☺

2. Guardar favorit (BD)

Primer, modifico *post.blade.php*:

```
@foreach($posts as $post)

    <!-- Mostro el títol del post -->
    <h2>{{ $post['title'] }}</h2>

    <!-- Mostro el contingut -->
    <p>{{ $post['body'] }}</p>

    <!-- Comprovo si aquest post ja està guardat com a favorit -->
    @if(in_array($post['id'], $favorits))

        <!-- Si ja és favorit, mostro aquest missatge -->
        <p><strong>Ja és favorit</strong></p>

    @else

        <!-- Si NO és favorit, mostro el formulari per guardar-lo -->
        <form method="POST" action="/favorit">

            <!-- Protecció CSRF obligatòria a Laravel -->
            @csrf

            <!-- Envio l'ID del post (amagat) -->
            <input type="hidden" name="post_id" value="{{ $post['id'] }}">

            <!-- Envio el títol del post -->
            <input type="hidden" name="title" value="{{ $post['title'] }}">

            <!-- Botó per guardar -->
            <button>Afegir a favorits</button>

        </form>

    @endif
```

Ara a *web.php* afegim ruta:

```
Route::post('/favorit', [PostController::class, 'store']);
```

Modifico *PostController.php*

```
    // Aquesta funció guarda un post com a favorit
    1 reference | 0 overrides
    public function store(Request $request)
    {
        // Creo un nou registre a la taula favorits
        Favorit::create([
            'post_id' => $request->post_id,
            'title' => $request->title
        ]);

        // Redirigeixo a la llista de posts
        return redirect('/posts');
    }
```

Per últim... *Favorit.php*:

```
class Favorit extends Model
{
    // Indico quins camps es poden omplir automàticament
    0 references
    protected $fillable = ['post_id', 'title'];
}
```

3. Mostrar quins són favorits

Per mostrar quins posts són favorits, es comparen els IDs dels posts amb els guardats a la base de dades.
A la vista Blade es mostra un missatge indicant si el post ja és favorit.

Et voilà!

4. Evitar duplicats

Última modificació a *PostController.php*

```
// Aquesta funció guarda un post com a favorit
1 reference | 0 overrides
public function store(Request $request)
{
    // Comprovo si ja existeix aquest favorit
    $existeix = Favorit::where('post_id', $request->post_id)->exists();

    // Si NO existeix, el guardo
    if (!$existeix) {
        Favorit::create([
            'post_id' => $request->post_id,
            'title' => $request->title
        ]);
    }

    // Redirigeixo a la llista de posts
    return redirect('/posts');
}
```

OPCIONAL (nota alta)

- eliminar favorits

Primer, modificaré de nou *posts.blade.php*:

```
<!-- Comprovo si aquest post ja està guardat com a favorit -->
@if(in_array($post['id'], $favorits))

    <!-- Si és favorit -->
    <p><strong>Ja és favorit</strong></p>

    <!-- Formulari per eliminar favorit -->
    <form method="POST" action="/eliminar-favorit">

        <!-- Protecció CSRF -->
        @csrf

        <!-- Envio l'ID del post -->
        <input type="hidden" name="post_id" value="{{ $post['id'] }}">

        <!-- Botó eliminar -->
        <button>Eliminar favorit</button>

    </form>
```

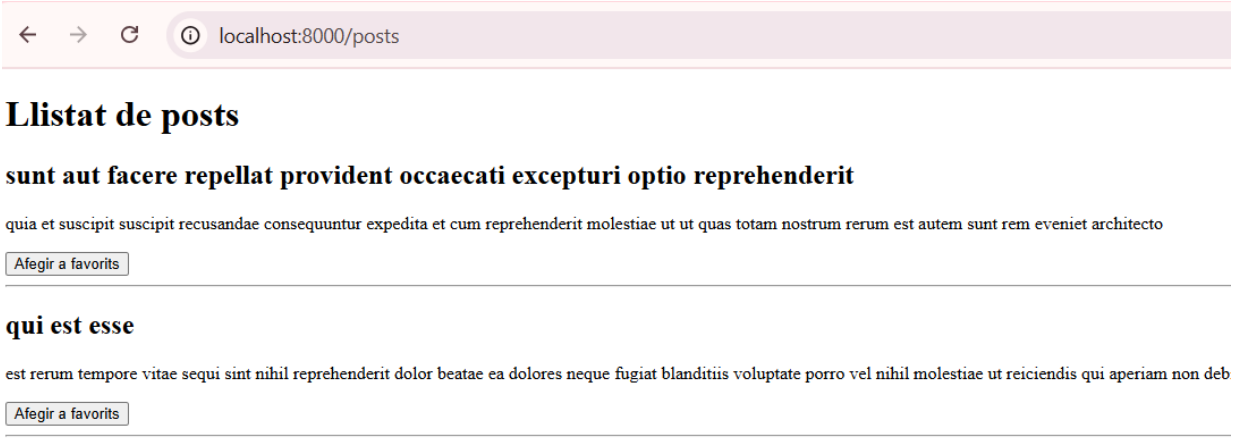
Afegeixo nova ruta a *web.php*:

```
// Ruta per eliminar un favorit
Route::post('/eliminar-favorit', [PostController::class, 'destroy']);
```

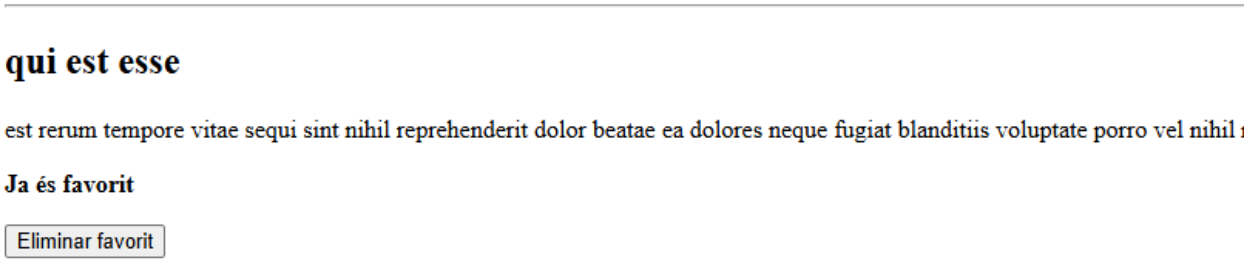
Ara toca la funció destroy:

```
// Aquesta funció elimina un favorit
1 reference | 0 overrides
public function destroy(Request $request)
{
    // Busco el favorit pel seu post_id i l'elimino
    Favorit::where('post_id', $request->post_id)->delete();

    // Torno a la llista
    return redirect('/posts');
}
```

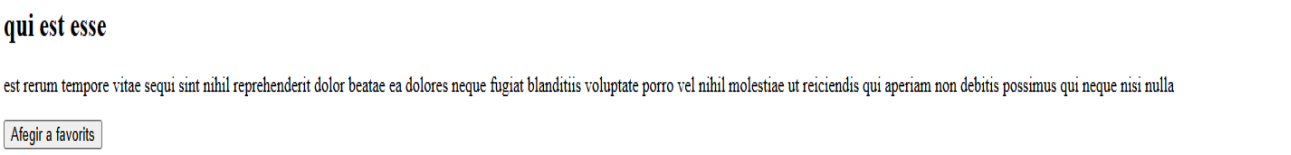
Ara anem a veure si realment he fet bé els canvis:



Ara l'afegeixo:



I ara l'elimino!



Fet 😊

- filtrar dades

Primer em toca fer una edició del *PostController.php*

```php
// Aquesta funció mostra només els posts que són favorits
0 references | 0 overrides
public function favorits()
{
    // Obtenim tots els favorits de la BD
    $favoritsBD = Favorit::pluck('post_id')->toArray();

    // Obtenim tots els posts de l'API
    $response = Http::get('https://jsonplaceholder.typicode.com/posts');
    $posts = $response->json();

    // Filtrarem només els que estan a favorits
    $postsFavorits = [];

    foreach ($posts as $post) {
        if (in_array($post['id'], $favoritsBD)) {
            $postsFavorits[] = $post;
        }
    }

    // Enviem només els favorits a la vista
    return view('favorits', compact('postsFavorits'));
}
```

Seguim amb *web.php*, afegint nova ruta:

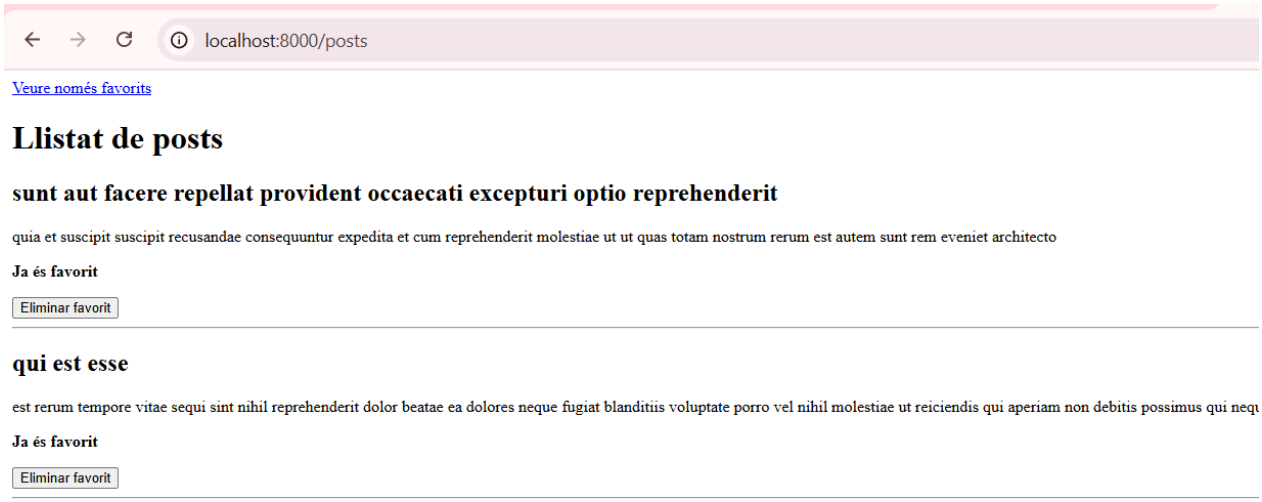
```php
// Ruta que mostra només els posts que estan marcats com a favorits
Route::get('/favorits', [PostController::class, 'favorits']);
```

Ara, toca crear una nova vista. Serà *favorits.blade.php* :

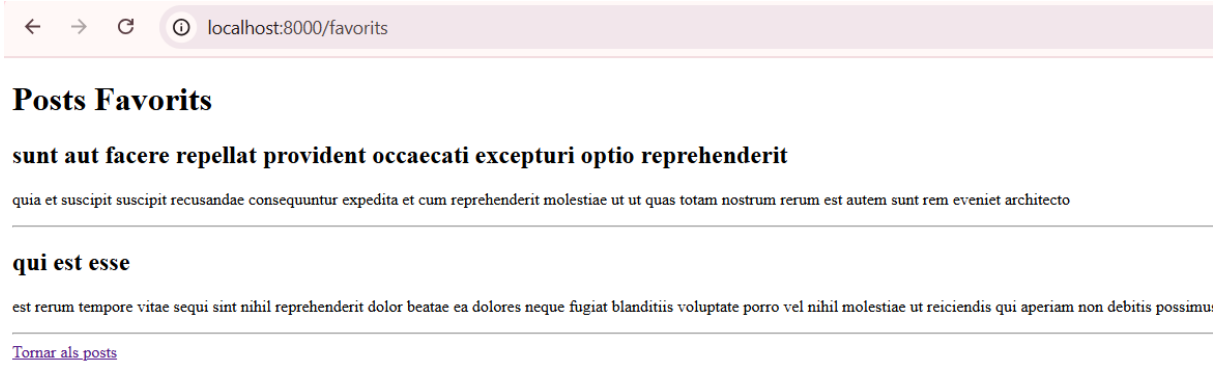
U05EX1 > consum-api-laravel > resources > views > favorits.blade.php

```html
<!DOCTYPE html>
<html lang="ca">

<head>
    <meta charset="UTF-8">
    <title>Favorits</title>
</head>

<body>

    <h1>Posts Favorits</h1>

    @foreach($postsFavorits as $post)
        <h2>{{ $post['title'] }}</h2>
        <p>{{ $post['body'] }}</p>
        <hr>
    @endforeach

    <a href="/posts">Tornar als posts</a>

</body>

</html>
```

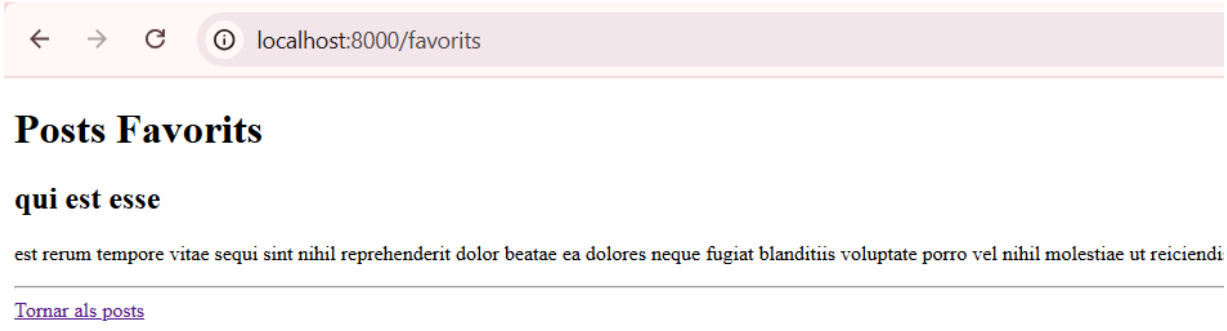
Ara vaig a fer les comprovacions pertinents:



Ara aniré a *http://localhost:8000/favorits*



Ara faré una prova eliminant un dels dos favorits:



Funciona!!

- UI millorada

Ara toca fer-ho una mica més maco :P

```
<h1>Llistat de posts</h1>

<!-- Enllaç per veure només els posts favorits -->
<a href="/favorits">Veure només favorits</a>

<hr>
```

Colorets:

```
@foreach($posts as $post)

    <!-- Contenidor visual per separar millor cada post -->
    <div style="border:1px solid #ccc; padding:10px; margin-bottom:10px;">

        <!-- Mostro el títol del post -->
        <h2>{{ $post['title'] }}</h2>

        <!-- Mostro el contingut -->
        <p>{{ $post['body'] }}</p>

        <!-- Comprovo si aquest post ja està guardat com a favorit -->
        @if(in_array($post['id'], $favorits))

            <!-- Si és favorit -->
            <p style="color: green;"><strong>Ja és favorit</strong></p>

            <!-- Formulari per eliminar favorit -->
            <form method="POST" action="/eliminar-favorit">

                <!-- Protecció CSRF -->
                @csrf

                <!-- Envio l'ID del post -->
                <input type="hidden" name="post_id" value="{{ $post['id'] }}">

                <!-- Botó eliminar amb color -->
                <button style="background-color: lightcoral;">Eliminar favorit</button>

            </form>

        @else
```

Aviam...

Ara faré el mateix amb /favorits:

```
<h1>Posts Favorits</h1>

<!-- Enllaç per tornar als posts -->
<a href="/posts">Tornar als posts</a>

<hr>

@foreach($postsFavorits as $post)

    <!-- Contenidor visual -->
    <div style="border:1px solid #ccc; padding:10px; margin-bottom:10px;">

        <!-- Títol -->
        <h2>{{ $post['title'] }}</h2>

        <!-- Contingut -->
        <p>{{ $post['body'] }}</p>

        <!-- Indicador visual -->
        <p style="color: green;"><strong>Favorit</strong></p>

    </div>

@endforeach
```

Ho comprovo:

localhost:8000/favorits

Posts Favorits

Tornar als posts

---

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

Favorit

qui est esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non deb

Favorit

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut

Favorit

ENTREGABLES

Codi font

- PHP (cURL)
- Laravel

Captures

- Postman (totes les operacions)
- aplicació funcionant

RÚBRICA D'AVALUACIÓ

RA7 — SERVEIS WEB (50%)

Criteri	Pes	Descripció
GET funcional	10%	recupera dades correctament
POST funcional	10%	envia JSON correctament
PUT funcional	10%	modifica dades
DELETE funcional	10%	elimina correctament
Ús correcte HTTP/JSON	10%	entén conceptes

RA9 — INTEGRACIÓ (50%)

Criteri	Pes	Descripció
Consum API en Laravel	10%	
Integració amb BD	15%	
Aplicació funcional	15%	
Codi estructurat	5%	
Documentació	5%	